



Región de Murcia
Consejería de Educación
y Formación Profesional



RMskills

17

DESARROLLO WEB TEST PROJECT

RMskills'25



**Cofinanciado por
la Unión Europea**

ÍNDICE

- INSTRUCCIONES GENERALES.
- PLAN DE PRUEBAS.
 - Definición de las pruebas
 - Programa de competición (horario y sesiones)
 - Esquema de clasificación (criterios de evaluación por módulos)
- MÓDULO (I)
 - Instrucciones de trabajo del módulo I (descripción de la prueba y duración)
 - Criterios de evaluación relacionados con el módulo I
 - Clasificación del módulo I
- MÓDULO (II)
 - Instrucciones de trabajo del módulo II (descripción de la prueba y duración)
 - Criterios de evaluación relacionados con el módulo II
 - Clasificación del módulo II.

1. INSTRUCCIONES GENERALES

La competición consistirá en el desarrollo de un trabajo práctico (**Test Project**) relacionado con el análisis, diseño y desarrollo de varios programa en Java que requerirá a los competidores poner en práctica sus conocimientos, habilidades y destrezas para demostrar sus competencias durante la prueba que deberá ser resuelta en un **tiempo máximo de 6,5 horas** distribuidas en **dos jornadas**.

Para ello dispones de un equipo con Windows 11, con el siguiente software:

- **Netbeans 8.2**
- **IntelliJ IDEA**

En Aula Virtual estás matriculado de un curso, **25-RMSkills**, con el que podrás acceder con las credenciales de murciaeduca (NRE / contraseña de murciaeduca). En el curso, dispones del

- **Material de apoyo:** curso de programación a distancia EaD en pdf y la API de Java. Deberás descargar el material de apoyo en tu equipo para su uso.
- **Tareas** en las que subir los diferentes módulos en los que está dividida la prueba.

No será posible la conexión a **internet** durante la prueba, a excepción del curso de Aula Virtual creado para el desarrollo de la misma.

Las credenciales para los equipos preparados para la prueba son:.

Login: rmskills

Password: rmskills2025@

2. PLAN DE LAS PRUEBAS

a. Definición de las pruebas

La competición consiste en la demostración de las competencias propias de programador a través de un trabajo práctico que pondrá de manifiesto la preparación de los competidores respecto a las siguientes competencias:

- Realiza el análisis, planificación y diseño de la aplicación solicitada.
- Realiza operaciones de entrada y salida de información
- Utiliza procedimientos específicos del lenguaje y librerías de clases.

- Uso eficiente de estructuras de control.
- Control y manejo adecuado del mecanismo de excepciones
- Escribe programas que manipulen eficientemente la información, seleccionando y utilizando tipos avanzados de datos.
- Desarrolla programas, aplicando características avanzadas de los lenguajes orientados a objetos.
- Incluye componentes de programación que maximicen la usabilidad.

La prueba está compuesta por **dos módulos**, y cada uno de ellos está formado por 3 ejercicios que, de manera progresiva, están orientados a la resolución de supuestos prácticos

b. Programa de competición (horario y sesiones)

La prueba se realiza en dos sesiones:

- **Primera sesión:** martes, **25 de marzo**, la hora de comienzo de la prueba es de **9:30** y finaliza a las **12:30**.
- **Segunda sesión:** miércoles, **26 de marzo** comienza a las **8:30** y finaliza a las **12:00**.

Ambas sesiones tienen las mismas condiciones descritas en los puntos anteriores.

c. Esquema de clasificación (criterios de evaluación por módulos)

TABLA DE GENERAL DE LA PRUEBA

Criterio	Calificación
Operaciones de entrada / salida	10%
Estructuras de control	25%
Excepciones	10%
Estructuras de datos	25%
Programación Orientada a Objetos	30%

En cada uno de los módulos, se encuentra la tabla de los criterios de calificación de cada uno de ellos.

Si la ejecución de cualquier programa terminase bruscamente por una **excepción** no controlada, la nota de ese ejercicio será de 0 puntos, independientemente de que, para alguno de los casos de prueba, la salida obtenida coincidiese con la salida esperada.

3. MÓDULO 1. Instrucciones de trabajo del módulo (descripción de la prueba y duración)

a. Descripción de la prueba

Juego Conecta 4

Es el clásico juego de estrategia donde 2 jugadores compiten por el control de la rejilla. Los jugadores escogen las fichas rojas o amarillas y las colocan en la rejilla por la parte superior, e intentan alinear 4 en horizontal, vertical o diagonal.

En el juego, la rejilla estará codificada por una matriz, y podrá tener, en cada una de sus celdas, uno de los tres valores (fichas de los jugadores) que codifican una situación:

- 1 - ficha del jugador 1,
- 2 - ficha del jugador 2,
- 0 - casilla vacía (no tiene ficha)

Ejercicio 1 - Conecta 4. Paso 1

- Dada una matriz concreta de 4x4, determina si uno de los jugadores, ha obtenido una línea horizontal, vertical o diagonal (4 fichas consecutivas del mismo jugador). Para ello, crea los siguientes métodos:
 - **leerMatriz()**: leerá los datos de la entrada estándar y almacena los valores en una matriz de tamaño 4x4.
 - **comprobarFila()**: comprueba si J1 o J2 ha conseguido 4 fichas seguidas en una fila.
 - **comprobarColumna()**: comprueba si J1 o J2 ha conseguido 4 fichas seguidas en una columna.
 - **comprobarDiagonalPrincipal()**: comprueba si J1 o J2 ha conseguido 4 fichas seguidas en la diagonal principal.
 - **comprobarDiagonalSecundaria()**: comprueba si J1 o J2 ha conseguido 4 fichas seguidas en la diagonal secundaria.
 - Para cada uno de los métodos, el programa debe mostrar un mensaje indicando qué jugador ha ganado.
- Solo puede haber **una línea de 4 fichas** (cuatro nº uno seguidos o cuatro nº dos seguidos) de un jugador (solo un ganador).
- En todos los casos de prueba habrá **un único ganador**.
- **No habrá errores** en ninguno de los datos de entrada, es decir, todos los casos son correctos.
- Se valorará de forma positiva la **eficiencia** del código y el uso de **programación modular**.

Ejemplo:

1	2	2	0
1	1	1	2
2	1	1	2
1	2	2	1

Ejercicio 2 - Conecta 4. Paso 2

- Adapta el ejercicio anterior para que se pueda jugar en una rejilla cuadrada de cualquier tamaño.
- Se valorará de forma positiva la eficiencia del código y el uso de programación modular..

Ejercicio 3 -Conecta 4. Paso 3

- Adapta el ejercicio anterior para que tenga en cuenta, las siguiente situaciones:
 1. La partida ha finalizado en tablas.
 2. La partida ha finalizado, con un ganador. Indica quién ha sido el ganador
 3. La partida no ha finalizado. Continúa la partida indicando quién es el jugador al que le toca jugar (J1 o J2), teniendo en cuenta que siempre comienza el J1. Continúa la partida hasta que uno de los jugadores obtenga una línea de 4 o se llegue a una situación de tablas (la rejilla está llena de fichas y no se ha conseguido ninguna fila de 4 fichas seguidas del mismo jugador)
- Se valorará de forma positiva la **eficiencia** del código y el uso de **programación modular**.

b. Criterios de evaluación relacionados con el módulo I y clasificación.

TABLA DE CALIFICACIONES

Ítem	%	Módulo 1			Módulo 2			Total
		Ejercicio 1.1	Ejercicio 1.2	Ejercicio 1.3	Ejercicio 2.1	Ejercicio 2.2	Ejercicio 2.3	
Entrada / Salida	10	2	4	4				10
Estructuras de control	25	2	7	7				25
Excepciones	10	-	-	-				10
Estructuras de datos	25	2	5	5				25
Programación Orientada a Objetos	30	-	-	-				30
Total	100	6	16	16				
Puntuación por módulo		38			62			100

En todos y cada uno de los ejercicios, se tendrá en cuenta la eficiencia del código, limpieza del mismo y el uso de programación modular

4. MÓDULO 2. Instrucciones de trabajo del módulo (descripción de la prueba y duración)

a. Descripción de la prueba

Estructura Trie

Ejercicio 1. Trie. Paso 1.

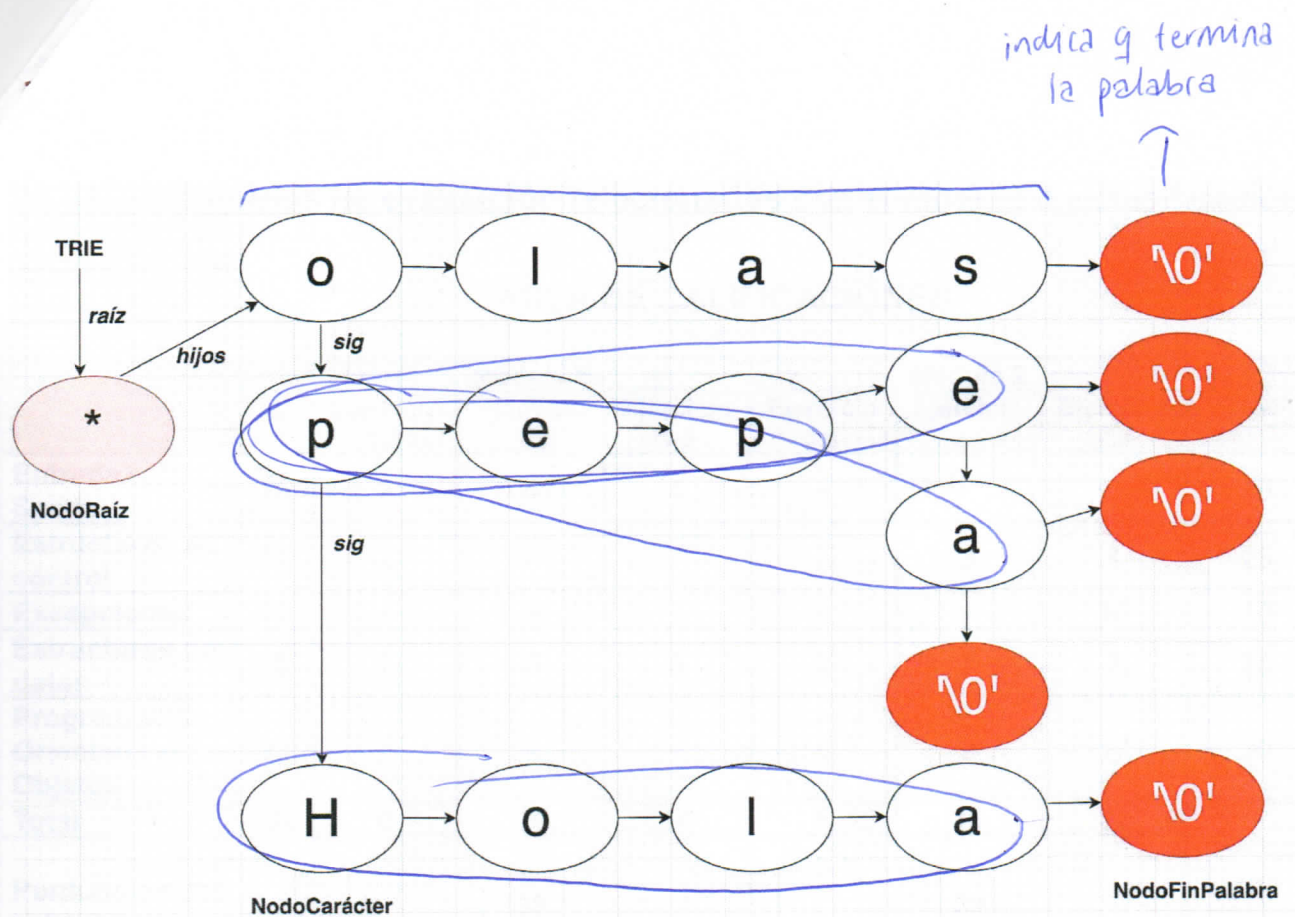
- Crea un programa que sea capaz de almacenar individualmente los caracteres de una palabra en una lista. Como estructura de datos para almacenar los caracteres, debes usar una de las clases que implementan la interfaz List, de la API de Java.

Ejercicio 2. Trie. Paso 2.

- Crea un programa que sea capaz de almacenar varias palabras. Como estructura de datos para almacenar los caracteres, debes usar una de las clases que implementan la interfaz List, de la API de Java.
- Cada palabra se guardará en una lista distinta.
- El programa debe implementar un método llamada *mostrarPalabras()* que muestre todas las palabras almacenadas.

Ejercicio 3. Trie. Paso 3.

- Cuando llegues a este ejercicio, debes solicitar al tutor, el diagrama de clases y la descripción de las mismas.
- Crea un programa que sea capaz de almacenar varias palabras, utilizando un sistema de listas múltiples. Esta estructura se suele llamar *Trie*. La siguiente imagen, muestra gráficamente la estructura de listas múltiples a crear.



- De acuerdo a la estructura anterior, las palabras almacenadas en dicho Trie son: olas, pep, pepe, pepa, Hola.

Duración

3 horas y media. De 8.30 a 12.00 horas.

b. Criterios de evaluación relacionados con el módulo II y clasificación

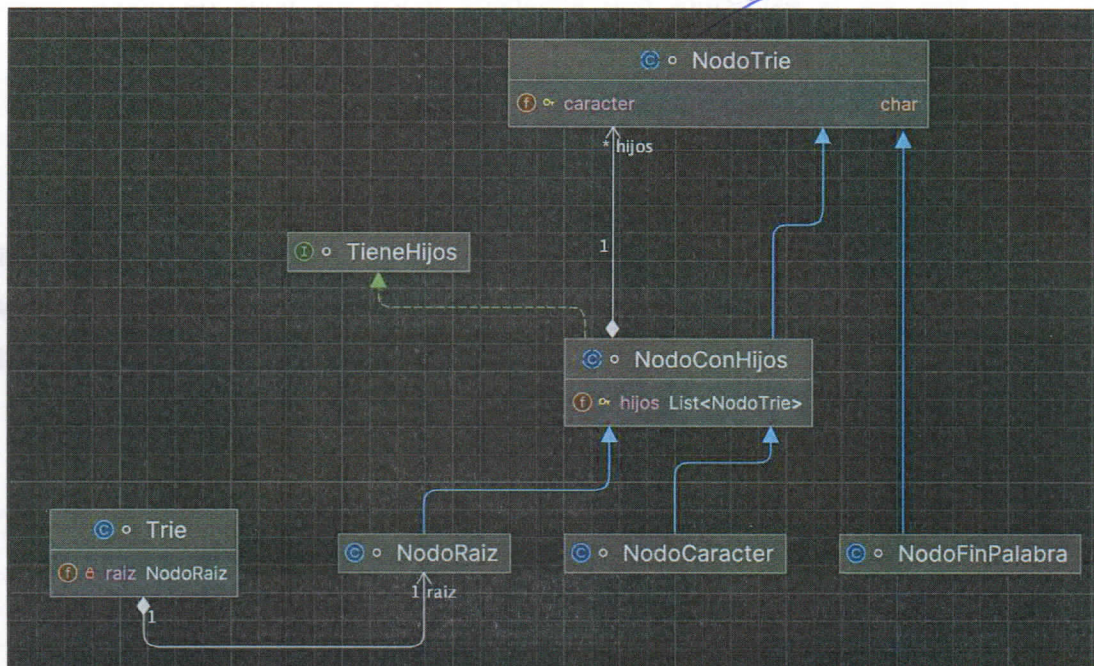
TABLA DE CALIFICACIONES

Ítem	%	Módulo 1			Módulo 2			Total
		Ejercicio 1.1	Ejercicio 1.2	Ejercicio 1.3	Ejercicio 2.1	Ejercicio 2.2	Ejercicio 2.3	
Entrada / Salida	10	2	4	4	-	-	-	10
Estructuras de control	25	2	7	7	2	3	4	25
Excepciones	10	-	-	-	-	-	10	10
Estructuras de datos	25	2	5	5	2	4	7	25
Programación Orientada a Objetos	30	-	-	-	4	9	17	30
Total	100	6	16	16	8	16	38	100
Puntuación por módulo		38			62			

En todos y cada uno de los ejercicios, se tendrá en cuenta la eficiencia del código y limpieza del mismo

Anexo del Ejercicio 3 - Módulo 2

- Este es el diagrama de clases correspondiente al ejercicio 3 del módulo 2



- Las clases que forman el diagrama son:
 - NodoTrie. Es la clase que define el nodo general. Solo tiene un atributo de tipo Char que almacena un carácter. Es una clase abstracta.
 - TieneHijos. Es una interfaz con los métodos:
 - public NodoTrie getHijo(char c)
 - public void agregarHijo(NodoTrie hijo)
 - NodoConHijos. Tiene como atributo una lista de nodos de tipo NodoTrie. Es la clase que se usa para agrupar las clases NodoRaiz y NodoCaracter bajo un tipo común. Es una clase abstracta.
 - NodoRaiz. Es la clase que representa al nodo principal de la estructura. A partir de él, se inician todas las palabras. Almacena siempre el carácter '*'.
 - NodoCarácter. Es la clase que representa al nodo fundamental de la estructura. Se

usan para almacenar por separado las palabras. Pueden almacenar cualquier carácter del abecedario español.

- **NodoFinPalabra.** Es la clase que representa al nodo que indica el final de una palabra. Aquellos prefijos que no terminan en un **NodoFinPalabra**, no se consideran palabras completas. Almacena siempre el carácter nulo ('\\0').
- **Trie.** Es la estructura que almacena todos los tipos de nodos. Implementa los siguientes métodos:
 - `public NodoTrie getRaiz()`
 - Devuelve el nodo raíz de la estructura.
 - `public void insertarPalabra(String palabra)`
 - Añade una nueva palabra a la estructura. Debe añadir solo las letras necesarias. Si por ejemplo quieres almacenar "pepa" y ya se encuentra "pepe" en la estructura trie, debes añadir solo la letra 'a', después del prefijo "pep".
 - `public boolean buscarPalabra(String palabra)`
 - Devuelve verdadero si la palabra se encuentra en la estructura y es una palabra completa, que tiene un **NodoFinPalabra** al final.
 - `public List<String> mostrarPalabras()`
 - Devuelve una lista de todas las palabras almacenadas en la estructura.